

Software Engineering

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Software Engineering — One-Liners	3
Introduction & Software Process	3
SDLC & Software Configuration Management	3
Software Development / Process Models	3
Requirement Engineering	4
Software Design	4
Software Metrics & Project Management	5
Software Testing	5
Software Maintenance & Reliability	6
Rapid-Fire Mixed Revision	7

Software Engineering — One-Liners

Introduction & Software Process

1. Software has **3** components: **programs**, **documentation** and **operating procedures**.
2. The **IEEE** definition of software engineering rests on **3** keywords: **systematic**, **disciplined** and **quantifiable**.
3. **Fritz Bauer (1968)** set **3** goals for software engineering: **economical**, **reliable** and **efficient** operation on real machines.
4. Software has **3** distinguishing characteristics: it is **developed not manufactured**, it **does not wear out**, and it is **mostly custom-built**.
5. Only **hardware** follows the **bathtub curve**; software shows **change-induced spikes** instead of a **wear-out** region.
6. **McCall's** quality model defines **11** factors grouped into **3** categories: **operation**, **revision** and **transition**.
7. McCall's **Product Operation** has **5** factors: **correctness**, **reliability**, **efficiency**, **integrity**, **usability**.
8. McCall's **Product Revision** has **3** factors: **maintainability**, **flexibility**, **testability**.
9. McCall's **Product Transition** has **3** factors: **portability**, **reusability**, **interoperability**.
10. **ISO 9126** lists **6** quality characteristics: **functionality**, **reliability**, **usability**, **efficiency**, **maintainability**, **portability**.
11. **FURPS** stands for **5** attributes: **Functionality**, **Usability**, **Reliability**, **Performance**, **Supportability**.
12. The term "**software crisis**" was popularised at the **1968 NATO** conference, which also coined "**software engineering**".
13. Software myths fall into **3** groups: **management**, **customer** and **practitioner/developer** myths.
14. **Brooks's Law** states **adding manpower** to a **late** project makes it **later**, refuting a **management** myth.
15. **Pressman's** generic process has **5** framework activities: **Communication**, **Planning**, **Modeling**, **Construction**, **Deployment**.
16. **Umbrella activities** like **risk management**, **SQA**, **SCM** and **measurement** span **all 5** framework activities.

SDLC & Software Configuration Management

17. The **SDLC** has **7** phases: **Planning**, **Requirement Analysis**, **Design**, **Coding**, **Testing**, **Deployment**, **Maintenance**.
18. The **costliest and longest** SDLC phase is **Maintenance**, often **60–70%** of total life-cycle cost.
19. Design splits into **2** levels: **HLD (High-Level/architectural)** and **LLD (Low-Level/module)** design.
20. A **baseline** can be changed only through **formal change-control procedures** after a **formal review**.
21. **SCM** comprises **5** tasks: **identification**, **version control**, **change control**, **configuration audit**, **status reporting**.
22. A **Software Configuration Item (SCI)** is any tracked work-product such as **code**, **SRS**, **design** or **test cases**.
23. An **ECO (Engineering Change Order)** defines a change's **constraints and review criteria** before it is made.
24. A **CCB/CCA (Change Control Board/Authority)** **approves or rejects** change requests in SCM.
25. Version control distinguishes a **variant** (functionally **equivalent** alternate) from a **revision** (a **superseding** change).
26. **Configuration audit** verifies a change was done **correctly**, while **status reporting** records **what changed, who, and when**.
27. Access is governed by **2** controls: **access control** (who may edit) and **synchronization control** (avoiding clashing edits).

Software Development / Process Models

28. The **Waterfall model** (Royce, **1970**) is **linear-sequential** and best suits **small, stable, well-defined** requirements.
29. The Waterfall's main weakness is **no working software until late** and **no tolerance for changing** requirements.
30. The **Incremental model** delivers in **builds** where the **first increment is the core product**.
31. **Prototyping** has **2** types: **throwaway/rapid** (discarded) and **evolutionary** (refined into the final product).
32. The biggest risk of prototyping is the **customer mistaking the prototype** for the **finished product**.
33. The **Spiral model** (Boehm, **1988**) is **risk-driven** and combines **iterative prototyping** with the **waterfall**.
34. Each Spiral loop has **4** quadrants: **objectives**, **risk analysis & resolution**, **develop/verify**, **plan next iteration**.
35. The **defining feature** of the Spiral model is **Risk Analysis**, making it ideal for **large, high-risk** projects.

36. The **RAD model** (James Martin) targets a **60–90 day** cycle using **reusable components** and **parallel teams**.
37. RAD has **5** phases: **Business, Data and Process Modeling, Application Generation, and Testing & Turnover**.
38. The **V-Model** pairs every **development (verification)** phase with a corresponding **testing (validation)** phase.
39. In the V-Model **Requirements ↔ Acceptance** testing and **System Design ↔ System** testing.
40. In the V-Model **HLD (architectural) ↔ Integration** testing and **LLD (module) ↔ Unit** testing.
41. The **Agile Manifesto (2001)** declares **4** values and **12** supporting principles.
42. Agile's **4** values prize **individuals, working software, customer collaboration** and **responding to change**.
43. **Scrum** defines **3** roles: **Product Owner, Scrum Master** and **Development Team**.
44. **Scrum** has **3** artifacts: **Product Backlog, Sprint Backlog** and **Increment**.
45. A Scrum **Sprint** is time-boxed to **2–4 weeks**, ending with a **Review** and a **Retrospective**.
46. The **Daily Scrum (stand-up)** is time-boxed to **15 minutes**.
47. **XP (Extreme Programming)** features **pair programming, TDD, continuous integration** and **refactoring**.
48. **XP** organises work into **4** activities: **planning, design, coding** and **testing**.
49. **Kanban's** signature practices are **visualising workflow** and **limiting Work-In-Progress (WIP)** with a **pull** system.
50. Model selection: **stable→Waterfall, high-risk→Spiral, short/modular→RAD, changing→Agile**.

Requirement Engineering

51. **Requirement Engineering** has **7** tasks: **Inception, Elicitation, Elaboration, Negotiation, Specification, Validation, Management**.
52. **Elicitation** techniques include **interviews, questionnaires, brainstorming, prototyping, FAST** and **QFD**.
53. **FAST = Facilitated Application Specification Technique**, a **joint** customer-developer meeting.
54. **QFD (Quality Function Deployment)** classifies requirements into **3** kinds: **normal, expected** and **exciting**.
55. Requirements split into **2** main types: **functional (what it does)** and **non-functional (how well)**.
56. **Non-functional** requirements are the "-ilities": **performance, security, usability, reliability, scalability**.
57. The **SRS** is governed by the **IEEE 830** standard and acts as the **customer-developer contract**.
58. A good SRS has **8** qualities: **correct, unambiguous, complete, consistent, verifiable, modifiable, traceable, ranked**.
59. **Unambiguous** means every SRS requirement has exactly **one** interpretation.
60. Requirement **validation** uses **reviews, prototyping, test-case generation** and **model validation**.
61. **Feasibility (TELOS)** covers **5** dimensions: **Technical, Economic, Legal, Operational, Schedule**.
62. **Economic feasibility** relies on **Cost-Benefit Analysis (CBA)** and **ROI** to justify the project.
63. Requirements are also seen as **user requirements** (high-level) versus **system requirements** (detailed/technical).

Software Design

64. Core design concepts include **abstraction, modularity, refinement, information hiding** and **functional independence**.
65. **Abstraction** has **2** levels: **procedural abstraction** and **data abstraction**.
66. **Modularity** applies **divide-and-conquer**, and an **optimal number of modules** minimises **total cost**.
67. **Stepwise Refinement** is a **top-down** strategy by **Niklaus Wirth**, complementary to **abstraction**.
68. **Information hiding** restricts each module's **internal decisions** behind a defined **interface**.
69. **Functional independence** is measured by **2** criteria: **high cohesion** and **low coupling**.
70. **Cohesion** is **intra-module** and should be **HIGH**; **coupling** is **inter-module** and should be **LOW**.
71. There are **7** cohesion types from worst to best: **coincidental, logical, temporal, procedural, communicational, sequential, functional**.
72. The **best cohesion is functional**; the **worst is coincidental**.
73. **Logical** cohesion groups same-class actions chosen by a **flag**; **temporal** cohesion groups actions by **timing**.
74. **Sequential** cohesion feeds **one element's output into the next**, just below **functional**.
75. There are **6** coupling types from worst to best: **content, common, external, control, stamp, data**.
76. The **worst coupling is content**; the **best (loosest) is data** coupling.

- 77. **Common** coupling shares **global data**; **control** coupling passes a **flag** that drives another module's logic.
- 78. **Stamp** coupling passes a **whole data structure** while using only **part** of it.
- 79. Good design demands **high cohesion** and **low coupling** simultaneously.
- 80. A **DFD** uses **4** symbols: **process (bubble)**, **data flow (arrow)**, **data store (open box)** and **external entity (square)**.
- 81. A **Level-0 DFD** is the **Context Diagram**, showing the **whole system as one process**.
- 82. A **DFD** shows **data flow only**, never **control flow**, **decisions**, **loops** or **timing**.
- 83. **Data dictionary** notation uses **= (composed of)**, **+** (and), **[]** (or), **{ }** (iteration), **()** (optional).
- 84. A **Structure Chart** marks a **data couple** with an **empty circle** and a **control couple** with a **filled circle**.
- 85. **Function-oriented** design is **top-down** using **DFDs**, while **object-oriented** design is built around **objects/classes**.
- 86. **UI design** follows **3** golden rules: **place the user in control**, **reduce memory load**, **make the interface consistent**.

Software Metrics & Project Management

- 87. **LOC** is a **size/direct** metric giving **productivity = KLOC/person-month** and **quality = defects/KLOC**.
- 88. LOC's chief drawback is being **language-dependent** and **unknown until late** in development.
- 89. **Function Points** were introduced by **Allan Albrecht (IBM, 1979)** and are **language-independent**.
- 90. FP uses **5** information-domain types: **EI, EO, EQ, ILF, EIF**.
- 91. FP adjustment uses **14** general system characteristics, each rated **0–5**.
- 92. The complexity factor is **CAF = 0.65 + 0.01 × ΣFi**, ranging from **0.65 to 1.35**.
- 93. The final count is **FP = UFP × CAF**, where **UFP** is the unadjusted total.
- 94. **COCOMO** was proposed by **Barry Boehm (1981)** to estimate **effort, time and cost** from **KLOC**.
- 95. COCOMO defines **3** modes: **Organic (<50 KLOC)**, **Semi-Detached (50–300 KLOC)**, **Embedded (>300 KLOC)**.
- 96. Basic COCOMO effort is **E = a × (KLOC)^b** person-months and time is **D = c × (E)^d** months.
- 97. Basic **Organic** constants are **a=2.4, b=1.05, c=2.5, d=0.38**.
- 98. The **Embedded** mode has the **highest exponent b = 1.20**; **Organic** the lowest at **b = 1.05**.
- 99. COCOMO has **3** levels: **Basic, Intermediate (15 cost drivers/EAF)** and **Detailed**.
- 100. **Intermediate COCOMO** multiplies effort by the **EAF** derived from **15** cost drivers in **4** groups.
- 101. A **Gantt chart** (Henry Gantt) is a **horizontal bar chart** of **tasks vs time** but is **weak on dependencies**.
- 102. **PERT** is **probabilistic/event-oriented** with **3** time estimates; **CPM** is **deterministic/activity-oriented** with **1**.
- 103. **PERT** expected time is **te = (to + 4tm + tp) / 6** using optimistic, most-likely and pessimistic times.
- 104. The **critical path** is the **longest** path yet equals the **shortest** possible project duration.
- 105. **Slack/Float** is the time an activity can slip without delaying the project; the **critical path has zero slack**.
- 106. **PERT** originated with the **US Navy (Polaris, 1958)**; **CPM** with **DuPont (1957)**.
- 107. **PERT** variance per activity is $\sigma^2 = ((tp - to)/6)^2$.
- 108. **Risk management (RMMM)** stands for **Risk Mitigation, Monitoring and Management**.
- 109. **Risk Exposure (RE) = Probability × Impact (cost)** of the risk.
- 110. Risks fall into **3** categories: **project, technical** and **business** risks.
- 111. **QA is process-oriented (prevention)** while **QC is product-oriented (detection)** through testing.

Software Testing

- 112. **Myers** defined testing as **executing a program to FIND errors**, so a **successful test uncovers an undiscovered error**.
- 113. **Dijkstra's** rule: testing shows the **presence** of defects, **never their absence**.
- 114. **Verification** asks **"are we building the product right?"**; **validation** asks **"are we building the right product?"**.
- 115. **Verification** checks against **specifications** (static + dynamic); **validation** checks against **user needs** (dynamic).
- 116. The defect chain is **Error → Fault/Defect/Bug → Failure**.
- 117. A **test case** has **3** parts: **inputs, execution conditions** and **expected results**.
- 118. A **test suite** is a **collection of test cases** executed together.
- 119. There are **4** testing levels in order: **Unit → Integration → System → Acceptance**.

120. **Unit testing** targets a **single module** in isolation, performed by the **developer** using **stubs and drivers**.
121. **Integration testing** verifies **interfaces** between combined modules.
122. **System testing** is **black-box** on the **whole system**, covering **recovery, security, stress and performance**.
123. **Acceptance testing** is done by the **customer** and includes **Alpha, Beta** and **UAT**.
124. **Regression testing** re-runs existing tests after a **change** to detect **newly introduced** defects.
125. There are **4** integration strategies: **big-bang, top-down, bottom-up** and **sandwich/hybrid**.
126. **Big-bang** integration combines **all modules at once**, making fault **localisation hard**.
127. **Top-down** integration uses **STUBS** to replace not-yet-ready **lower** modules.
128. **Bottom-up** integration uses **DRIVERS** to replace not-yet-ready **calling** modules.
129. A **stub** is **called by** the module under test; a **driver calls** the module under test.
130. **Stubs $\Rightarrow$ Top-Down; Drivers $\Rightarrow$ Bottom-Up** — the most common testing MCQ pairing.
131. **Sandwich/Hybrid** integration combines **top-down and bottom-up** around a **target layer**.
132. **White-box testing** is also called **structural, glass-box** or **clear-box** testing.
133. White-box coverage has **4** criteria: **statement, branch/decision, condition** and **path** coverage.
134. **Branch coverage subsumes statement coverage**, and **path coverage** is the **strongest**.
135. **Cyclomatic Complexity** was given by **Thomas McCabe (1976)** to count **independent paths**.
136. Cyclomatic complexity has **3** equal formulas: **$V(G) = E - N + 2P = D + 1 = \text{number of regions}$** .
137. In **$V(G) = E - N + 2P$** , **E** = edges, **N** = nodes and **P** = connected components (usually **1**).
138. **$V(G) = D + 1$** uses **D** = number of **decision/predicate** nodes.
139. **McCabe** recommends keeping **$V(G) \leq 10$** per module.
140. **Basis path testing** uses **$V(G)$** as the **number of independent test paths** to design.
141. **Control-flow testing** uses the flow graph; **data-flow testing** uses **definition-use (def-use)** chains.
142. **Black-box testing** is also called **functional** or **behavioural** testing, based on **requirements only**.
143. **Equivalence Partitioning** divides inputs into **valid and invalid classes**, testing **one** representative each.
144. **Boundary Value Analysis (BVA)** tests at **range edges** (**a-1, a, a+1 ... b-1, b, b+1**) where defects cluster.
145. A **Decision Table** maps **combinations of conditions** to **actions**.
146. **Cause-Effect Graphing** links **causes (inputs)** to **effects (outputs)** and converts to a **decision table**.
147. **Alpha testing** is at the **developer's site** with the developer **present**; **Beta** is at the **user's site** with developer **absent**.
148. Both **Alpha** and **Beta** testing are forms of **acceptance** testing.
149. **Static testing** finds defects **without executing code**, making it a **verification** activity.
150. A **Formal Technical Review (FTR)/peer review** is the umbrella covering **walkthroughs** and **inspections**.
151. A **Walkthrough** is **informal** and **led by the author**, with few formal rules.
152. An **Inspection (Fagan)** is the **most formal** review, **led by a moderator (not the author)** with a **checklist** and **follow-up**.
153. A formal inspection has **5** roles: **moderator, author, reader, recorder (scribe)** and **inspector**.
154. **Code inspection** and **standards compliance** are static checks against a **defect checklist** and **coding standards**.

Software Maintenance & Reliability

155. **Software maintenance** is modification **after delivery** and is the **longest, costliest** life-cycle phase.
156. There are **4** maintenance types (**CAPP**): **Corrective, Adaptive, Perfective, Preventive**.
157. **Corrective** maintenance **fixes reported bugs**, accounting for roughly **21%** of effort.
158. **Adaptive** maintenance handles a **changed environment** (OS, hardware, law), roughly **25%**.
159. **Perfective** maintenance **adds/enhances features** and is the **largest** at roughly **50%**.
160. **Preventive** maintenance improves future **maintainability** and is the **smallest** at roughly **4%**.
161. **Forward engineering** flows **requirements $\rightarrow$ design $\rightarrow$ code** in the normal direction.
162. **Reverse engineering** recovers **design/specs from existing code** without changing it (**design recovery**).
163. **Re-engineering** = **reverse engineering plus forward engineering** to produce an **improved** system.

- 164. **Re-engineering** has 5 steps: **inventory analysis, document restructuring, reverse engineering, code/data restructuring, forward engineering.**
- 165. **Reliability** is the **probability of failure-free operation** for a **specified time** in a **specified environment.**
- 166. **MTTF** = mean time **to failure**; **MTTR** = mean time **to repair.**
- 167. **MTBF = MTTF + MTTR** is the most-tested reliability relationship.
- 168. **Availability** = $[\text{MTTF} / (\text{MTTF} + \text{MTTR})] \times 100\%$, i.e. **uptime ÷ total time.**
- 169. **Lehman's Laws** include **Continuing Change** and **Increasing Complexity** of evolving software.

Rapid-Fire Mixed Revision

- 170. The **first increment** in incremental development always delivers the **core product.**
- 171. The **3 "core"** feasibility types tested most are **Technical, Economic** and **Operational.**
- 172. The **defining quadrant** of the Spiral model is **risk identification and resolution** (quadrant **2**).
- 173. A **"respond within 2 seconds"** clause is a **non-functional (performance)** requirement.
- 174. **Functional cohesion** and **data coupling** together represent the **ideal** modular design.
- 175. **Content coupling** (worst) lets one module **alter another's internal data** directly.
- 176. **Coincidental cohesion** (worst) groups elements with **no meaningful relationship.**
- 177. **Basic-Organic a = 2.4** but **Intermediate-Organic a = 3.2** — the constants differ by level.
- 178. A **fault** that is **never executed** can **never** produce a **failure.**
- 179. **BVA** specifically targets **off-by-one** and **boundary** errors.
- 180. **Equivalence partitioning** mainly **reduces the number** of test cases needed.
- 181. The **critical path** determines the **minimum project duration** despite being the **longest** route.
- 182. A **Gantt chart** excels at showing **timelines/overlap** but poorly shows **task dependencies.**
- 183. **Walkthrough = author-led/informal**; **Inspection = moderator-led/formal** — both are **static.**
- 184. **Y2K-style** fixes are **Adaptive**; adding a **new report** is **Perfective**; fixing a **crash** is **Corrective.**
- 185. **Verification** is mostly **static (reviews)**; **validation** is mostly **dynamic (execution).**
- 186. The **V-Model** plans **testing early**, in **parallel** with development phases.
- 187. **Information hiding** plus **modularity** together enable **low coupling.**
- 188. **Path coverage** is the **most exhaustive** white-box criterion and the **costliest.**
- 189. The **number of independent paths** equals the **cyclomatic complexity V(G).**
- 190. **RAD** demands **modular systems, reusable components** and **highly skilled** developers.
- 191. **Agile** values **working software** over **comprehensive documentation.**
- 192. **SCM** applies across **all** SDLC phases as an **umbrella** activity, not a single phase.
- 193. **Software does not wear out**, but it **deteriorates** because each **change** adds new defects.
- 194. **Quality control (QC)** detects defects in the **product**; **quality assurance (QA)** improves the **process.**
- 195. A **prototype** is most useful when requirements are **unclear or fuzzy.**
- 196. **System testing** verifies **non-functional** aspects like **stress, security and performance.**
- 197. **Acceptance testing** validates the system against the **user's actual requirements.**
- 198. The **SRS** is produced in the **requirement specification** task and reviewed in **validation.**
- 199. **Maintainability, flexibility and testability** make up McCall's **product revision** group.
- 200. **MTBF rises** as reliability **improves**, while **availability** depends on **both MTTF and MTTR.**
- 201. A **baseline** becomes official **only after** a **formal technical review**, then needs **change control** to modify.
- 202. **Refactoring** improves **internal structure** while keeping **external behaviour unchanged.**